

Southeastern European Regional Programming Contest
Bucharest, Romania
October 15, 2011

Problem A
Secret — MAX

Input File: A.IN
Output File: standard output
Program Source File: A.C, A.CPP, A.JAVA

Agent Smith transfers to a secret center secret information consisting of one strictly positive integer number. For coding the number, he presents each digit of the number by its corresponding word, and concatenates the words. For example, `42` may be transformed to `FOURTWO`. Agent Smith knows four languages (the 1st, the 2nd, the 3rd and the 4th). In these languages, numbers from `0` to `9` are spelled as follows (all letters are capital Latin, ASCII codes in range `65-90`):

Digit	Language 1 spell	Language 2 spell	Language 3 spell	Language 4 spell
0	ZERO	SIFIR	ZERO	ZERO
1	UNU	BIR	JEDEN	ONE
2	DOI	IKI	DWA	TWO
3	TREI	UC	TRZY	THREE
4	PATRU	DORT	CZTERY	FOUR
5	CINCI	BES	PIEC	FIVE
6	SASE	ALTI	SZESC	SIX
7	SAPTE	YEDI	SIEDEM	SEVEN
8	OPT	SEKIZ	OSIEM	EIGHT
9	NOUA	DOKUZ	DZIEWIEC	NINE

After such encoding, he adds to the code some “noise” — redundant capital Latin chars. Any chars (including the chars already used in the code) can be added in any quantities in any positions of the code.

Write a program which should read the code and search the largest possible number coded by that code, making two different assumptions:

1. We don’t know what language Smith used, but it was the same language (of the given) overall the code;

2. Smith could code different digits using either different languages (of the given) or the same.

Your program should input all lines and process them separately. Each single line contains not more than 10^5 capital Latin chars — the code. Size of the input is less than `1 Mb`.

For each line from the input file your program should output two single-space-separated integers — the largest numbers according to the listed two assumptions in that order.

It’s guaranteed, that each code was received from a correct positive (non-zero) integer. By the way, it means that both results will be always positive. Your program should output them without leading zeroes.

Sample input	Sample output
SIZEROXSIEVDEMX	**66 67**

Problem B
Safety Grade

Input File: B.IN
Output File: standard output
Program Source File: B.C, B.CPP, B.JAVA

The sites of a cable network are interconnected by cables such that a cable connects a single pair of distinct sites, and a pair of sites can be connected by several cables. We say that the network is connected if any two sites in the network are directly or indirectly connected; otherwise the network is disconnected. The safety grade **s** of the network is defined as follows.

- **s** is `0` if the network is disconnected, or the number of sites is `0` or `1`.
- If the number of sites is greater than `1` then **s** is the minimum number of cables that disconnect the network when removed, i.e. removing any `s-1` cables keeps the net connected, while the removal of some **s** cables disconnects the net.

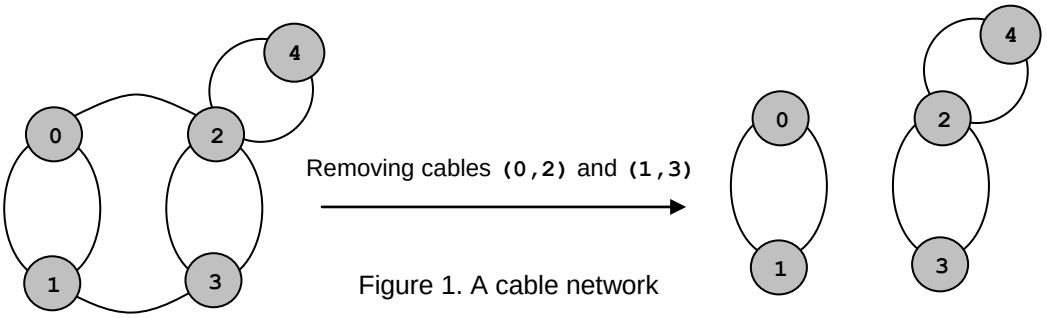


Figure 1. A cable network

For example, consider the net in figure 1, where the sites correspond to the shadowed circles and the cables are indicated by lines. The network stays connected when removing any single cable, whereas the removal of cables `(0,2)`, and `(1,3)` disconnects the net. Another variant for disconnecting the net is by removing the cables `(2,4)` and `(2,4)`. The safety grade is `s=2`.

Write a program that reads several data sets from a text file and computes the safety grade of the cable networks the data sets encode. Each data set starts with two integers: the number `0≤n≤100` of sites in the net, and `0≤m≤1000` the number of cables in the net. Follow `m` data pairs `(u,v)`, `u<v`, where `u` and `v` are site identifiers (integers from `0` to `n-1`). A pair `(u,v)` designates a cable that interconnects the sites `u` and `v`. The pairs may occur in any order. Except the `(u,v)` pairs, which do not contain white spaces, white spaces can occur freely in input. Input data terminate with an end of file and are correct. For each data set, the program prints on the standard output, from the beginning of a line, the safety grade of the encoded net.

Table 1. Input and output samples

Input	Output
`0 0`	`0`
`2 1 (0,1)`	`1`
`2 0`	`0`
`5 8 (0,1) (1,3) (2,3) (0,2) (0,1) (2,3) (2,4) (2,4)`	`2`

An input/output sample is given in table 1. The first data set encodes an empty network. The second data set encodes a net with `2` sites and `1` cable. The third data set corresponds to a disconnected network with two sites. The fourth data set encodes the net shown in figure 1.

Problem C
Search by template

Input File: C.IN
Output File: standard output
Program Source File: C.C, C.CPP, C.JAVA

The pattern `P` and the text `T` are given. They contain the letters of the Latin alphabet (lowercase and uppercase). The pattern may also contain the characters `'?'`, `'['`, `']'`, `'{'` and `'}'` You must find all the positions where `P` appears in `T`.

Each position in the pattern `P` may contain:

- a letter from the Latin alphabet (`'a' – 'z', 'A' – 'Z'`)
- the character `'?'` that can replace any letter (on its place can be any letter)
- brackets `'['...']'`, between them there is the set of letters that may be in the current position
- brackets `'{'...'}'`, between them there is the set of letters that are not allowed to be in the current position
- the letters can be repeated in brackets like [`asssa`] or {`kLLf`}.

For example, if the pattern `P` is `A?[bcCc]{De}`, then the first letter of a match must be `A`, the second can be any Latin letter, the third - letter `b`, `c` or `C`, and the fourth can be any Latin letter except `D` or `e`.

Input. The first line contains the number of test cases *n*. Each test consists of two lines. The first line contains the pattern `P` with no more than `100` characters and `60` positions. The second line contains the text with no more than 10^6 letters.

Output. For each test case print on a separate line all text positions where the pattern matches, in ascending order (the first letter from the text T is assumed to have the position '1'). If the pattern `P` does not appear in text `T`, print the message "`no match`". There must be a single space between each printed position, and no extra space at the beginning/end of line.

Sample input	Sample output
3 A?[bcCc]{De} yAqCpsApbeAocqq ???[QWERTY] aSdFrQererRTY {eRT}? eRTeRTq	2 11 3 8 9 10 no match

Problem D
Roots Intervals

Input File: D.IN
Output File: standard output
Program Source File: D.C, D.CPP, D.JAVA

Consider the function $f(x)=\tan(\sin(x))-\sin(\tan(x))+\cos(x)^5-0.5$ defined on the interval $[a,b]$, and $nb \geq 1$ a series of subintervals $[x_i, x_{i+1}]$, $i=1,nb$, where $x_1=a$ and $x_{nb+1}=b$. Find the number of subintervals that contain "observable" roots of $f(x)$. A root in a subinterval $[x_i, x_{i+1}]$ is "observable" if the existence of that root can be decided without inspecting the behavior of $f(x)$ for $x_i<x<x_{i+1}$, i.e. a subinterval is a black box and you cannot compute inside the interval.

The program input is from a text file. Each data set in the file stands for a particular interval $[a,b]$ of $f(x)$ and specifies the limits a, b (real numbers) and the integer number nb of subintervals. For each data set the program prints the number of subintervals that contain "observable" roots of $f(x)$. Each result is printed on the standard output from the beginning of a line.

White spaces can occur freely in the input. The input data are correct and terminate with an end of file. An input/output sample for the function $f(x)=1-x^2$ is in the table bellow.

Input	Output
-2 2 2 0 100 5 -1 1 1	2 1 1

Problem E
Palindromic Dates

Input File: E.IN
Output File: standard output
Program Source File: E.C, E.CPP, E.JAVA

A date is said to be palindromic if, when written in the form "YearMMDD", the result is a palindrome (a number that has the same value regardless if we read it left-to-right or right-to-left). For example, the date "January 1st, 1" is palindromic since it would be written as "10101"; "December 1st, 1021" is palindromic too, since it would be written as "10211201".

Given a date, you are required to find the next valid date (in chronological order) that is palindromic (according to the definition of "palindromic date" that was given in the previous paragraph). For the purpose of this problem, a date is considered valid if the year is between 1 and 2147483647, the month is between 1 and 12, the day is at least 1 and:

- for months 1,3,5,7,8,10,12 – the day is not greater than 31
- for months 4,6,9,11 – the day is not greater than 30
- for month 2, the day is not greater than 29, and can be 29 only during leap years. A year is considered "leap year" if it is divisible by 400, or is divisible by 4 but not by 100.

The input file contains one date per line: thee numbers separated by the character '/', representing (in order) the day, month and year. For example, July 4th 2011 would be written as "04/07/2011" (or 4/07/2011, or 4/7/2011). The last line in the input file is empty.

The output file shall contain the next valid palindromic date for each date in the input file, one date per line. The date format in the output file is DD/MM/Year – i.e. the separator character is '/', the year's first digit must never be 0, and you must always use two digits for day and month (use 01 instead of 1, 02 instead of 2 etc.).

It is guaranteed that all input dates are valid, and that for all input dates there is a "next valid palindromic date".

Sample input	Sample output
1/1/1 12/11/1 01/06/138048967	01/02/1 11/12/1 31/08/138048984



Problem F
Post Offices

Input File: F.IN
Output File: standard output
Program Source File: F.C, F.CPP, F.JAVA

You are given the location of all post offices on earth, and a set of destinations for various packages that need to be shipped. For each package, you need to find the maximal set of post offices that are suitable as destinations for the package – i.e. the set of post offices must satisfy the following constraints:

- The post office that is closest to the final destination must be in the result set.
- All post offices that are at most `2%` farther than the closest post office must also be present in the result set.

For the purpose of this problem, the earth is approximated to be a perfectly flat sphere, with the radius 6381 Km. The distance between two points may be computed using the haversine formula:

*A = sin((lat1-lat2)/2) ^ 2 + cos(lat1) * cos(lat2) * (sin((long1-long2)/2) ^ 2) ;*

*DISTANCE = 6381 * 2 * atan2(sqrt(A), sqrt(1-A));*

(you may use other formulas if you please, but note that the arccosine formula has large rounding errors for small distances, and thus you may get wrong results).

The input file is a binary file; all numbers written in it are written in little-endian representation. It contains, in this order:

- `4` bytes that represent the number of post offices `N` – integer number, no greater than `3000000`
- For each post office, the input file contains two numbers`<LAT LONG>` (`4` bytes each) representing the latitude and longitude (in degrees) of the post office. The numbers are single-precision IEEE754 floating point numbers. All latitudes are between `-90` and `90` degrees; all longitudes are between `-180` and `180` degrees.
- `4` bytes representing the number of packages `M` (integer number, no greater than 500000)
- For each package, the input file contains two numbers `<LAT LONG>` (`4` bytes each) representing the latitude and longitude (in degrees) of the destination. The numbers are single-precision IEEE754 floating point numbers. All latitudes are between `-90` and `90` degrees; all longitudes are between `-180` and `180` degrees.

The output file is also a binary file, containing only 32-bit integers written in little-endian representation. For each of the `M` lines in the input file, it must contain:

- One integer – the number `P` of post offices in the destination set
- `P` integers – the indices of the post offices in the destination set, sorted in ascending order (lower indices first). The index of the first post office in the input file is considered to be `1` (and thus, all indices should be between `1` and `N`, inclusively)

Sample input	Sample output
N/A – binary file	**N/A – binary file**



Southeastern European Regional Programming Contest
Bucharest, Romania
October 15, 2011

Problem G
Choosing a camera

Input File: G.IN
Output File: standard output
Program Source File: G.C, G.CPP, G.JAVA

Mr. Strange is choosing a compact digital photo camera. He looks at various shops offers and sometimes makes temporary choices. At each temporary choice, he knows about all offers he already saw, but doesn't know about those he will see in the future.

According to Mr. Strange, any digital camera is fully described by two properties: the number of pixels and the optical zoom ratio. For each property, he thinks that the more is the better. Mr. Strange is afraid to buy an outdated camera, so he never chooses a camera, if he knows about another camera, which is strictly better by any one property and not worse by another. Among all cameras Mr. Strange does not consider outdated, he chooses the cheapest one; if there are different non-outdated cameras with equal minimal cost, he chooses among them the oldest offer.

The first line of input contains the number of the test cases. The first line of each test case contains total number n ($1 \le n \le 10^5$) of actions. Each action can be either a new camera offer or a temporary choice. Each offer is denoted by big letter `P`, then camera's number of pixels, zoom ratio and cost. Each temporary choice is denoted by big letter `C`. Each action is denoted in separate line, data inside lines are separated by single spaces.

Numbers of pixels are integers in the range $10^3 \le p_i \le 10^9$. Zoom ration is a floating-point number in the range $1 \le z_i \le 100$, with no more than 6 digits after decimal point. Costs are integers in the range $1 \le c_i \le 10^6$. Size of the input is less than 16 Mb.

For each temporary choice action, your program should output the number of the action when the currently chosen camera was offered. If no choice can be done according to the rules, the query result should be −1. All results inside each test case should be written in the same line and separated by single spaces. Results for consecutive test cases should be in consecutive lines.

Sample input	Sample output
2 1 P 9600000 7 72 7 P 1200000 1 40 P 7200000 5 100 C P 9600000 3 200 C P 7200000 12 220 C	2 2 4

The first line of the output is empty, because first test case contain no `C` actions. At the first choosing in second test case, offer 1 (1200000 1 40) is outdated, so 2 (7200000 5 100) is the best because the only. At the 2nd, the same choice, because it's cheaper than 4 (9600000 3 200). At the 3rd, 6 (7200000 12 220) makes 2 (7200000 5 100) outdated, so 4 (9600000 3 200) becomes the cheapest among non-outdated.



Southeastern European Regional Programming Contest
Bucharest, Romania
October 15, 2011

Problem H
Road Planner

Input File: H.IN
Output File: standard output
Program Source File: H.C, H.CPP, H.JAVA

An acyclic road network is composed of unidirectional road segments (edges), each connecting two intersections (vertices) out of a total of N intersections. The time necessary to travel on a road segment is always positive and it depends linearly on the number of cars C using the segment s:

$$T(s) = a_s C + b_s$$

a_s and b_s are two constants expressed as single precision floating point numbers, describing the properties of road segment s. The time necessary to cross an intersection is 0.

A number of cars must travel from vertex 0 to vertex $N-1$ in this road network. Each car chooses its route selfishly in order to reduce its own travel time; each car knows that all the other cars will also selfishly choose their route. You are asked to write a program that computes how many cars will travel on each of the possible paths from the source to the destination, and the time needed to travel on those paths.

The input file contains several tests and is organized as follows. The first line contains the number of tests. The following lines contain the specification of the tests. Each test contains on the first line the number of vertices, the number of edges and the number of cars, separated by whitespaces. After that, each edge is given on a separate line and contains the following fields separated by spaces: source vertex, destination vertex, a_s and b_s
The output will contain one line for each test input, specifying the minimum time for any car to travel in the network, rounded down to the nearest integer.

Example Input:	**Example output:**
2 4 4 4000 0 1 0.01 0 0 2 0 45.1 1 3 0 45.1 2 3 0.01 0 4 5 4000 0 1 0.01 0 0 2 0 45.1 1 3 0 45.1 1 2 0 0 2 3 0.01 0	65 80

The first input consists of 4 vertices and 4 edges; 4000 cars must travel from vertex 0 to vertex 3. To achieve the smallest travel time, cars will choose one of the two possible paths $(0-1-3)$ and $(0-2-3)$ such that the number of cars on both is equal; hence minimum travel time is lower $(0.01 * 2000 + 45.1) = 65$.

The second input is similar to the first one, except for a zero cost edge that is added between vertices 1 and 2. In this case, all cars selfishly choose to take the path $(0-1-2-3)$ and the minimum travel time is 80.



Southeastern European Regional Programming Contest
Bucharest, Romania
October 15, 2011

Problem I
Term Generator

Input File: I.IN
Output File: standard output
Program Source File: I.C, I.CPP, I.JAVA

A formula has the syntax shown in figure 1(a). If a formula has the particular syntax given in figure 1(b) we say that the formula is in Normal Form (NF).

```
<formula>::= <variable> |
             (+<formulae>) | (*<formulae>)
<variable>::= a lower case letter from the English
              alphabet
<formulae>::= <formula> |
              <formula><formulae>
```

a) The general syntax of a formula

```
<NF_formula>::= <term> | (+<terms>)
<term>::= <variable> | (*<variables>)
<terms>::= <term><term> | <term><terms>
<variables>::= <variable><variable> |
               <variable><variables>
```

b) The syntax of a NF formula

Figure 1. Formula syntax

A formula is converted to NF according to the rewriting rules given below, where **F** represents a formula, **S** stands for a non empty sequence of formulae, and **s** and **s'** denote possibly empty sequences of formulae. Applying a rewriting rule **q→r** on a formula **F** means to substitute by **r** a part of **F** that matches the pattern **q**, as shown in figure 2. The conversion terminates when no rewriting rule can be applied. The conversion terminates for any formula, and the result is unique regardless which rules are applied on which parts of the formula and in which order.

1. **(+F)→F**
2. **(*F)→F**
3. **(+s(+S)s')→(+sSs')**
4. **(*s(*S)s')→(*sSs')**
5. **(*s(+FS)s')→(+(*sFs')(*s(+S)s'))**

```
(+(*(+(*ab)(+a))b))      -1->
(+(*(+(*ab)a)b))         -5->
(+(+(*(*ab)b)(*(+a) b))) -1->
(+(+(*(*ab)b)(*ab)))     -4->
(+(+(*abb)(*ab)))        -1->
(+(*abb)(*ab))
```

Figure 2. Converting a formula to NF

Let **NF(F)** be the normal form of the formula **F**. The problem is to write a term generator that for a given formula **F** and a number **k** outputs the next **k** terms from **NF(F)** in the order in which they appear in **NF(F)**. If the terms are exhausted, the generator continues from the first term of **NF(F)**. For example, consider **F=(+(*(+(*ab)(+a))b))**, and **NF(F)=(+(*abb)(*ab))** as in figure 2. Generating the first term from **NF(F)** yields **(*abb)**. Generating two more terms produces **(*ab)**, **(*abb)**. Notice that if **NF(F)** contains similar terms, as in the last example in figure 3, these terms are considered distinct.

Write a term generator that reads sets of data from the standard input. The content of a data set is **F** $k_1 \ldots k_n$ **0**, **n>0**, where **F** is a formula, and $k_1 \ldots k_n$ are long integers different than **0**. For each k_i, **i=1,n**, the program generates the next $|k_i|$ terms from **NF(F)** and, if k_i**>0**, prints these terms on the standard output. Each printed term starts from the beginning of a line and there are no white spaces between the characters of the term. The generated terms are not printed if k_i**<0**. White spaces are used freely in the input. Each formula **F** in the input has at most **150** characters and each term of **NF(F)** is at most **80** characters long, not counting white spaces. The input data terminate with an end of file, and are correct.

Input	Output
(+(*(+(*ab)(+a))b)) -3 1 1 0 (+xyy) -2 1 0	(*ab) (*abb) y

Figure 3. Input/output sample

Problem J
Painting

Input File: J.IN
Output File: standard output
Program Source File: J.C, J.CPP, J.JAVA

There is a wall consisting of nxn tiles. Long ago those tiles were painted in k different colors. Now the paint wears out and the wall has to be repainted. This time all the tiles of the wall should be painted in just one of the k colors. In one move we can paint one horizontal or one vertical row of tiles. Also we can paint a row in a given color only if at least two tiles in this row (horizontal or vertical) are already in this color (either old or new paint). You need to find out the minimal number of moves needed to paint all tiles.

The first line of input contains the number of test cases. The first line of each test case contains two integers: n $(1 < n \leq 500)$ – the number of tiles in one row and k $(1 \leq k < n)$ – the number of different colors available. In each of the following n lines there are n natural numbers giving the numbers of the colors in which the corresponding tile was painted before. The colors have the numbers from 1 to k.

The first line of the output for each test case should contain the number q – the minimal number of moves needed to paint all the tiles. In the second line list the numbers of all the colors in which we can paint the wall according to the given rules with the same number of moves. The numbers should be listed in increasing order. If it's impossible to paint all the tiles following the rules in the statement then print just the number 0.

Sample input	Sample output
2 **3 2** **1 2 1** **2 1 1** **1 2 2** **2 1** **1 1** **1 1**	**4** **1** **2** **1**